

DD: FUL-02-STEP-CAPTURE — Order Capture Entry

Developer implementation guide: DD_FUL-02-STEP-CAPTURE-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_FUL-02-STEP-CAPTURE-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md The first step in the order journey: a channel UI (Field Agent PWA, Backoffice UI, Call Center UI, self-service portal) hands FUL-02 the customer's intent to subscribe; FUL-02 turns that into a durable order row and starts the operator's workflow running in Camunda. Phase: CAPTURED.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-STEP-CAPTURE-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /engine-rest/process-definition/key/OrderCapture_WIK
- POST /api/orders
- POST /engine-rest/process-definition/key/OrderCapture_WIK/start

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;

- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- main-start
- order-validate
- validate-order

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- PaymentApplied
- PaymentAppliedConsumer

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.

Concern	Recommendation
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md

The first step in the order journey: a channel UI (Field Agent PWA, Backoffice UI, Call Center UI, self-service portal) hands FUL-02 the customer's intent to subscribe; FUL-02 turns that into a durable order row and starts the operator's workflow running in Camunda. Phase: CAPTURED.

1. What this step does

A customer at a known HomePass wants to subscribe to a Package. The capturing channel — the screen the human (agent or self-service customer) is interacting with — sends an HTTP POST to `POST /api/orders`. This step's job is to turn that single HTTP call into two durable things, atomically from the channel's point of view:

1. **A row in `customer_order`** (the DB table where the order lives for its whole lifetime, holding identifiers, status, audit). This row is what the rest of the system reads when it needs to know about this order: the BO UI, finance reports, NOC dashboards, ops investigations.
2. **A live workflow execution in Camunda** for this order. Camunda is the workflow engine — it lives in its own server outside our module and runs the operator's BPMN flowchart that defines what happens to an order from capture to activation. The execution is called a **process instance** — one running execution of a recipe. Starting it is how the rest of the journey is launched: from this moment on, Camunda will drive the order through validation, payment, install, activation, by handing work to FUL-02's workers as the flow advances.

The two things are bound together. The DB row knows about the Camunda process instance via the `process_instance_id` column; the Camunda process instance knows about the DB row via its `businessKey` (Camunda's way of tagging an instance with the domain entity it belongs to — here, the `orderId`). If a Kafka consumer or REST call later needs to wake the Camunda instance — for example, when a payment arrives — it'll look up the order by some external reference, get the `process_instance_id` or `businessKey`, and call Camunda's REST API.

If either of the two writes fails after the other succeeded, the step rolls back: if Camunda fails to start, the DB row is deleted. The channel sees a 5xx and can retry safely thanks to the **idempotency key** — a string the channel generates per submission (e.g., from the device ID + timestamp) so that re-sending the same request never creates two orders.

Why a separate "capture" step at all

The other steps in FUL-02 are Camunda-driven: Camunda decides when they run and hands them work. Capture is the **only** step that runs **before** Camunda is involved — it's the bootstrap. It's the bridge between HTTP-from-a-channel and a running Camunda instance. Once capture completes, the rest of the journey is Camunda-orchestrated.

Customer-side

The customer record already exists in the customer service when the agent is at the door (KYC and identity were captured during the sales conversation). In v1.0 the customer service is stubbed by the `customer_stub` table (the ILM-CFG-01 Customer Service DD is pending). The channel passes the `customerId` of the existing record. `customer_stub` is a deliberate v1.0 placeholder — its columns are documented in FRAMEWORK §6.1; it will be retired when ILM-CFG-01 lands.

2. Worked example end-to-end (WIK shape)

This walks one concrete capture from agent tap to 201 response.

Scenario: Wednesday 18 May 2026, 09:00 EAT. Field agent `agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1` is at customer John Mwangi's home in Karen, Nairobi. He's just collected ID documents and signed paperwork; the customer wants the `pkg_INET_VOIP_RES` triple-play package. The agent taps "Submit Order" on the iPhone 15 PWA.

Step 1 — PWA POSTs to FUL-02

The PWA generates an `idempotencyKey` from its device ID + timestamp + sequence counter — guaranteed unique per submission, identical if the agent taps Submit twice or the device retries on network failure. The PWA sends:

```
POST /api/orders HTTP/1.1
Host: fulfillment-order-capture.sophix.internal
Authorization: Bearer eyJhbGc... (agent JWT with role ORDER_FIELD_AGENT)
Content-Type: application/json
Content-Length: 412

{
  "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "operatorCode": "WIK",
  "packageRef": "pkg_INET_VOIP_RES",
  "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "salesChannel": "FIELD_AGENT_PWA",
  "salesAgent": "agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "idempotencyKey": "fapwa-iphone15-20260518T090000-001",
  "intendedActivationDate": null,
  "capturedAt": "2026-05-18T09:00:00Z",
  "metadata": {
    "capturedLocation": { "lat": -1.319, "lng": 36.706 },
    "deviceId": "iphone15-FA001",
    "notes": "Saturday install preferred"
  }
}
```

Step 2 — FUL-02 validates and looks up idempotency

Inside `OrderCaptureService`:

- Authenticates the JWT; role `ORDER_FIELD_AGENT` is allowed for `salesChannel = FIELD_AGENT_PWA`.
- Validates the envelope: all required fields present, `operatorCode = WIK` is in the allowed set, `capturedAt` is not in the future, `idempotencyKey` is non-empty.
- Looks up `customer_order` by (`operator_code = 'WIK'`, `sales_channel = 'FIELD_AGENT_PWA'`, `idempotency_key = 'fapwa-iphone15-20260518T090000-001'`). No match. This is a genuinely new order.

Step 3 — FUL-02 checks the operator's BPMN is deployed

The process definition key for WIK is `OrderCapture_WIK`. FUL-02 asks Camunda whether that BPMN is currently deployed:

```
GET /engine-rest/process-definition/key/OrderCapture_WIK HTTP/1.1
Host: camunda.sophix.internal
Authorization: Basic <module-svc-creds>
```

Response 200 — the BPMN is deployed:

```
{
  "id": "OrderCapture_WIK:5:abc-def-ghi",
  "key": "OrderCapture_WIK",
  "version": 5,
  "deploymentId": "dep_01HZ...",
  "suspended": false
}
```

(If the BPMN had not been deployed, the response would have been 404 and FUL-02 would have returned 400 `BPMN_NOT_DEPLOYED_FOR_OPERATOR` to the PWA without writing a DB row.)

Step 4 — FUL-02 generates the orderId and writes the customer_order row

FUL-02 generates a ULID for the order: `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`. The `payment_reference` column is set to the same value (convention used later by STEP-PAYMENT's `PaymentAppliedConsumer` as a fallback lookup key when a payment arrives but `subscription_id` is not yet known — see STEP-PAYMENT §3.1). INSERT:

```
INSERT INTO customer_order (
  order_id, customer_id, operator_code, package_ref, homepass_id,
  sales_channel, sales_agent, idempotency_key, payment_reference,
  intended_activation_date, process_definition_key,
  current_phase, current_activity_id,
  created_at, created_by, last_updated_at, metadata
) VALUES (
  'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1',
  'cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6',
  'WIK', 'pkg_INET_VOIP_RES', 'hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P',
  'FIELD_AGENT_PWA', 'agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1',
  'fapwa-iphone15-20260518T090000-001',
  'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1',
```

```

    NULL, 'OrderCapture_WIK',
    'CAPTURED', 'main-start',
    '2026-05-18 09:00:00.123', 'agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1',
    '2026-05-18 09:00:00.123',
    '{"capturedLocation":{"lat":-1.319,"lng":36.706},"deviceId":"iphone15-FA001","notes":"Saturday install preferred"}':js
);

```

process_instance_id is left NULL for now (Camunda hasn't been asked yet).

Step 5 — FUL-02 starts the Camunda process instance

FUL-02 now asks Camunda to start running the operator's flow for this order. `businessKey = orderId` tags the new instance so we can find it again later:

```

POST /engine-rest/process-definition/key/OrderCapture_WIK/start HTTP/1.1
Host: camunda.sophix.internal
Authorization: Basic <module-svc-creds>
Content-Type: application/json

{
  "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "variables": {
    "orderId": { "value": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1", "type": "String" },
    "customerId": { "value": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6", "type": "String" },
    "operatorCode": { "value": "WIK", "type": "String" },
    "packageRef": { "value": "pkg_INET_VOIP_RES", "type": "String" },
    "homepassId": { "value": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P", "type": "String" },
    "salesChannel": { "value": "FIELD_AGENT_PWA", "type": "String" },
    "salesAgent": { "value": "agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1", "type": "String" },
    "capturedAt": { "value": "2026-05-18T09:00:00Z", "type": "String" },
    "idempotencyKey": { "value": "fapwa-iphone15-20260518T090000-001", "type": "String" }
  }
}

```

Camunda returns 200:

```

{
  "id": "cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "definitionId": "OrderCapture_WIK:5:abc-def-ghi",
  "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "ended": false,
  "suspended": false
}

```

The process instance is now alive in Camunda. Internally, Camunda has already advanced from the BPMN's start event to the first external task (order-validate, owned by STEP-VALIDATE) and is offering it for FUL-02's workers to pick up via long-poll.

Step 6 — FUL-02 stores the process_instance_id and emits OrderCreated

```

UPDATE customer_order
  SET process_instance_id = 'cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1',
      current_activity_id = 'validate-order',
      last_updated_at     = NOW()
 WHERE order_id           = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';

```

`current_activity_id` is updated to `validate-order` because the `PhaseProjector` (FRAMEWORK §4 — the listener that syncs Camunda's runtime state onto the DB row) has by now seen Camunda advance past `main-start`. `current_phase` is recomputed: per the `WIK activity_phase_map`, `validate-order` → `IN_VALIDATION`.

FUL-02 emits the order-created event on Kafka:

```

{
  "eventType": "OrderCreated",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T09:00:00.345Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "packageRef": "pkg_INET_VOIP_RES",
    "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "processDefinitionKey": "OrderCapture_WIK",
    "processInstanceId": "cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "salesChannel": "FIELD_AGENT_PWA",
    "salesAgent": "agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "createdAt": "2026-05-18T09:00:00.123Z"
  }
}

```

Step 7 — FUL-02 returns 201 to the PWA

HTTP/1.1 201 Created
Content-Type: application/json

```
{
  "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "processDefinitionKey": "OrderCapture_WIK",
  "processInstanceId": "cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "currentPhase": "IN_VALIDATION",
  "currentActivityId": "validate-order",
  "createdAt": "2026-05-18T09:00:00.123Z"
}
```

The agent's PWA shows "Order submitted ✓"; the agent walks off to the next customer. Total elapsed inside this step: ~250 ms. Camunda meanwhile is offering order - validate to FUL-02's worker pool — the next step (STEP-VALIDATE) is already underway.

3. API endpoint reference

POST /api/orders

Request — body fields

Field	Type	Required	Plain-English meaning	Example
customerId	string (ULID)	yes	The customer this order is for. Verified to exist in ILM-CFG-01 (v1.0 stub: customer_stub). Customer creation happens out-of-band before order capture (KE flow: agent's CRM tooling at sales-conversation time).	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
operatorCode	enum	yes	Which operator's BPMN to run. One of WIK / WUG / WTZ / YASN. Maps to the BPMN OrderCapture_{operatorCode}. v1.0 ships OrderCapture_WIK only.	WIK
packageRef	string	yes	Package the customer is subscribing to. Validated by STEP-VALIDATE against SIP-01.	pkg_INET_V0IP_RES
homepassId	string (ULID)	yes	HomePass where install will happen. Validated by STEP-VALIDATE against RLM-CFG-01.	hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P
salesChannel	enum	yes	Which channel this came from. v1.0 KE accepts FIELD_AGENT_PWA / BACKOFFICE / CALL_CENTER. Must match the JWT's role (rule R-FUL-02-FW-1).	FIELD_AGENT_PWA
salesAgent	string (ULID)	yes	The human who captured the order. Used for commission attribution, audit.	agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
idempotencyKey	string	yes	Channel-supplied dedup key. Must be unique per (operator_code, sales_channel, idempotency_key). PWAs typically derive this from device ID + timestamp + sequence.	fapwa-iphone15-20260518T090000-001
intendedActivationDate	date (ISO 8601)	no	If the customer wants service activated on a specific future date. Interpretation depends on the BPMN (see rule R-FUL-02-CAP-5). NULL = immediate.	2026-06-01 or NULL

Field	Type	Required	Plain-English meaning	Example
capturedAt	datetime (ISO 8601 UTC)	yes	When the customer signed (channel's clock). Used for commission timing, ops SLA dashboards. Rejected if in the future.	2026-05-18T09:00:00Z
metadata	object	no	Channel-specific payload. Goes into customer_order.metadata JSONB column.	{"capturedLocation":{"lat":-1.319,"lng":36.706},"deviceId":"iphone15-FA001"}

Responses

Status	When it fires	Meaning
201 Created	All checks passed; order row written; Camunda instance running.	The order is captured and the journey has started. Body contains orderId, processInstanceId, current phase.
400 Bad request — BPMN_NOT_DEPLOYED_FOR_OPERATOR	The resolved process key OrderCapture_{operatorCode} has no deployed BPMN definition in Camunda.	Ops misconfig: a new operator was added to the allowed set but their BPMN wasn't deployed. No DB row is written. Channel should not retry until ops resolves.
400 Bad request — INVALID_ENVELOPE	Required field missing, capturedAt in the future, channel-role mismatch, etc.	Channel bug or stale token. Channel should not retry the same payload.
403 Forbidden — CHANNEL_ROLE_MISMATCH	JWT's role doesn't permit salesChannel. E.g., a backoffice user POSTing with salesChannel = FIELD_AGENT_PWA.	Auth violation; logged for security review.
409 Conflict — idempotency hit	An order already exists for the same (operator_code, sales_channel, idempotency_key).	Channel re-submitted (network retry, etc.). Response returns the existing order so the channel can resync state.
500 Internal server error	Camunda start failed after the order row was written, and the compensating delete also failed (rare).	The order row may be orphaned (no process_instance_id). Ops investigates; channel may retry with the same idempotencyKey and get the same orderId.

Response 201 body:

```
{
  "orderId": "ord_01HZ...A",
  "processDefinitionKey": "OrderCapture_WIK",
  "processInstanceId": "cam_pi_01HZ...",
  "currentPhase": "IN_VALIDATION",
  "currentActivityId": "validate-order",
  "createdAt": "2026-05-18T09:00:00.123Z"
}
```

Response 400 body:

```
{
  "error": "BPMN_NOT_DEPLOYED_FOR_OPERATOR",
  "detail": "No BPMN deployed for operator WIK. Contact ops.",
  "operatorCode": "WIK",
  "expectedProcessDefinitionKey": "OrderCapture_WIK"
}
```

Response 409 body:

```
{
  "orderId": "ord_01HZ...A",
  "processDefinitionKey": "OrderCapture_WIK",
  "currentPhase": "ACTIVATED",
  "idempotencyHit": true
}
```

4. Service flow inside FUL-02

OrderCaptureService runs the entry transaction. The numbered steps below are what the worked example in §2 walks concretely.

1. **Authenticate.** Verify JWT, extract role. Channel-role pairing per FRAMEWORK R-FUL-02-FW-1 (ORDER_FIELD_AGENT ↔ FIELD_AGENT_PWA, ORDER_BACKOFFICE ↔ BACKOFFICE, ORDER_CALL_CENTER ↔ CALL_CENTER). Mismatch → 403.

2. **Validate envelope.** Required fields present; operatorCode ∈ {WIK, WUG, WTZ, YASSN} (KE v1.0 deployment accepts WIK only at runtime — see step 5; the schema allows others for forward-compat); capturedAt not in future; idempotencyKey non-empty. Fails → 400 INVALID_ENVELOPE with detail field.
3. **Idempotency check.** SELECT FROM customer_order WHERE (operator_code, sales_channel, idempotency_key) matches. Hit → return existing order with idempotencyHit: true (409). The existing order's phase is returned as-is — if the customer's first submission already activated, the channel sees currentPhase: "ACTIVATED".
4. **Resolve customer.** customerId is required (channel-supplied). Verified to exist in customer_stub (v1.0); future-state verified against ILM-CFG-01.
5. **Verify BPMN is deployed.** GET /engine-rest/process-definition/key/OrderCapture_{operatorCode}. 404 → return 400 BPMN_NOT_DEPLOYED_FOR_OPERATOR. **Critical:** do NOT write the order row if the BPMN is missing. The order would be orphaned (no flow to run it through).
6. **Generate orderId; INSERT customer_order.** ULID for order_id. payment_reference = order_id (a convention used by STEP-PAYMENT's PaymentAppliedConsumer to correlate inbound PaymentApplied events back to the order — the customer types order_id as the M-Pesa account number when paying). current_phase = CAPTURED, current_activity_id = main-start. process_instance_id left NULL for now.
7. **Start the Camunda process instance.** POST /engine-rest/process-definition/key/OrderCapture_{operatorCode}/start with businessKey = orderId and process variables. See FRAMEWORK §1.1 for the full call shape.
8. **UPDATE customer_order.process_instance_id.** The Camunda response carries the new instance id; store it on the row.
9. **Emit** sophix.orders.created on Kafka. See §6 below.
10. **Return 201.**

Compensating transaction on partial failure

If step 7 fails after step 6 succeeded (Camunda 500, network error, etc.), the order row exists but no process instance does. OrderCaptureService issues a DELETE on the row, then returns 5xx to the channel. The channel retries with the same idempotencyKey; the second attempt's idempotency check (step 3) won't match because the row was deleted, so it proceeds fresh.

If step 8 fails (the Camunda call succeeded but the UPDATE failed), there's a brief inconsistency — Camunda has an instance whose businessKey doesn't appear on any DB row. A startup reconciler in OrderCaptureService scans for process_instance_id IS NULL AND created_at < NOW() - INTERVAL '5 minutes' rows and queries Camunda by businessKey to backfill.

5. DB tables touched

customer_order (FRAMEWORK §5.1) — INSERT then UPDATE

STEP-CAPTURE writes the row on initial INSERT, then UPDATES process_instance_id once Camunda confirms the start.

Columns set by this step:

Column	When set	Example value
order_id	INSERT	ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
customer_id	INSERT	cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
operator_code	INSERT	WIK
package_ref	INSERT	pkg_INET_V0IP_RES
homepass_id	INSERT	hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P
sales_channel	INSERT	FIELD_AGENT_PWA
sales_agent	INSERT	agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
idempotency_key	INSERT	fapwa-iphone15-20260518T090000-001
payment_reference	INSERT	ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1 (= order_id)

Column	When set	Example value
intended_activation_date	INSERT	2026-06-01 or NULL
process_definition_key	INSERT	OrderCapture_WIK
process_instance_id	UPDATE after Camunda start	cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
current_phase	INSERT, then UPDATE by PhaseProjector	CAPTURED initially → IN_VALIDATION after Camunda advances
current_activity_id	INSERT, then UPDATE by PhaseProjector	main-start initially → validate-order after Camunda advances
created_at	INSERT	2026-05-18 09:00:00.123
created_by	INSERT	agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1 (or customer_self_portal)
last_updated_at	INSERT, then UPDATED on every change	2026-05-18 09:00:00.123
metadata	INSERT	{"capturedLocation":{"lat":-1.319,"lng":36.706},"deviceId":"iphone15-FA001","notes":"Saturday install preferred"}

Sample data — rows captured by this step at five points in time, before any later steps modify them:

order_id	operator_code	package_ref	sales_channel	sales_agent	idempotency_key	current_phase	current_activity_id	process_instance_id	created_at
ord_01HZ...A	WIK	pkg_INET_VOIP_RES	FIELD_AGENT_PWA	agent_01HZ...A	fapwa-iphone15-20260518T090000-001	IN_VALIDATION	validate-order	cam_pi_01HZ...A	2026-05-18 09:00:00.123
ord_01HZ...B	WIK	pkg_INET_100M_ONLY	BACKOFFICE	bo_user_42	bo-20260518T093000-xyz	IN_VALIDATION	validate-order	cam_pi_01HZ...B	2026-05-18 09:30:15.402
ord_01HZ...C	WIK	pkg_INET_VOIP_RES	CALL_CENTER	cc_agent_07	cc-20260518T094500-021	IN_VALIDATION	validate-order	cam_pi_01HZ...C	2026-05-18 09:45:33.871
ord_01HZ...D	WIK	pkg_INET_500M_PRO	FIELD_AGENT_PWA	agent_01HZ...D	fapwa-iphone15-20260518T100000-002	IN_VALIDATION	validate-order	cam_pi_01HZ...D	2026-05-18 10:00:00.512
ord_01HZ...E	WIK	pkg_TV_HFC_BASIC	BACKOFFICE	bo_user_17	bo-20260518T102000-005	IN_VALIDATION	validate-order	cam_pi_01HZ...E	2026-05-18 10:20:11.339

customer_order_phase_history (FRAMEWORK §5.2) — first row appended

PhaseProjector (the Camunda history listener) appends the very first transition row for the order when the instance enters main-start:

Column	Example value
id	ordh_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
order_id	ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
operator_code	WIK
previous_phase	NULL
new_phase	CAPTURED
previous_activity_id	NULL
new_activity_id	main-start
transition_at	2026-05-18 09:00:00.123
triggered_by	system_startup
triggered_by_detail	OrderCaptureService
metadata	{"salesChannel":"FIELD_AGENT_PWA","salesAgent":"agent_01HZ...A","idempotencyKey":"fapwa-iphone15-20260518T090000-001"}

A second row follows immediately (within ~200 ms) as Camunda advances to validate-order:

Column	Example value
id	ordh_01HZX9K8M7Q2N4P6R3T5W8Y0Z2
order_id	ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
previous_phase	CAPTURED
new_phase	IN_VALIDATION
previous_activity_id	main-start
new_activity_id	validate-order
transition_at	2026-05-18 09:00:00.345
triggered_by	worker_completion
triggered_by_detail	camunda_engine_advance
metadata	NULL

customer_stub (FRAMEWORK §6.1)

Not written by this step in v1.0 KE flow — the channel always passes an existing customerId. The stub row exists already (created out-of-band by the field agent's CRM tooling at sales-conversation time; see ILM-CFG-01 pending DD for the future customer-creation flow).

6. Kafka events emitted

sophix.orders.created

Emitted on the topic `sophix.orders.created` immediately after `OrderCaptureService` completes the Camunda start and the row UPDATE. Carries enough context that downstream consumers don't need to look up the order to react.

```
{
  "eventType": "OrderCreated",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T09:00:00.345Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "packageRef": "pkg_INET_VOIP_RES",
    "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "processDefinitionKey": "OrderCapture_WIK",
    "processInstanceId": "cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "salesChannel": "FIELD_AGENT_PWA",
    "salesAgent": "agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "createdAt": "2026-05-18T09:00:00.123Z"
  }
}
```

Consumers and what they do with it:

Consumer	Purpose
CVM (Customer Value Management)	Logs the order as a top-of-funnel acquisition event. Drives the dashboard "orders captured today by sales channel × operator".
Finance ETL	Forecasting pipeline: an order captured today is a probable activation in 1–7 days; finance models revenue accordingly.
NOC alerting	Counts capture rate per channel per hour. Sudden drops trigger an investigation (channel down? PWA bug? OPS outage?).

Note: DD_NOT-01 does NOT consume `OrderCreated`. Customer-facing notifications start at `OrderAwaitingPayment` (STEP-PAYMENT) when there's something for the customer to act on. Sending an SMS at capture time would just spam customers.

7. Step-pinned rules

ID	Rule
R-FUL-02-CAP-1	Idempotency unique on (operator_code, sales_channel, idempotency_key). Hit → return the existing order (with whatever phase it has now) and idempotencyHit: true. Channels are expected to dedupe network retries via this key.
R-FUL-02-CAP-2	The customer_order row is written *before* Camunda is asked to start; process_instance_id is UPDATED onto the row after Camunda confirms. businessKey = orderId links the two.
R-FUL-02-CAP-3	If OrderCapture_{operatorCode} is not deployed in Camunda, return 400 and do NOT write the order row. An order without a flow to run would be orphaned.
R-FUL-02-CAP-4	payment_reference is initialized to order_id at capture. STEP-PAYMENT's PaymentAppliedConsumer uses this convention to correlate inbound PaymentApplied events to the order: the customer is instructed to type order_id as the M-Pesa account number; BIL-01 echoes that as targetReference in the PaymentApplied event; FUL-02's consumer matches it against payment_reference.
R-FUL-02-CAP-5	intendedActivationDate interpretation is BPMN-defined. WIK BPMN v1.0 ignores it (immediate activation). A future operator BPMN supporting scheduled activations would set a timer firing install creation at intendedActivationDate - ful_02_config.future_activation_lead_days. WIK v1.0 BPMNs treat the date as advisory metadata only.
R-FUL-02-CAP-6	The channel passes an existing customerId (KE v1.0 flow). v1.0 verifies it exists in customer_stub; future-state verifies against ILM-CFG-01 with same customerId semantics. Customer creation is out of v1.0 FUL-02 scope.
R-FUL-02-CAP-7	DB-write-then-Camunda-start is a compensating-transaction pair. If Camunda start fails after the row was written, the row is DELETED; channel retries with the same idempotencyKey and succeeds fresh. A startup reconciler covers the rare case where the row was UPDATED partially (Camunda started, but the process_instance_id UPDATE failed).
R-FUL-02-CAP-8	The OrderCreated event is emitted only after both writes succeed (row + Camunda). Consumers can trust the event: when they see it, both DB and Camunda are consistent.